Unit 6: Hash Tables (Week 5)

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

Part 1: The Map ADT

Section 6.1

🤔 You need to look up a student's GPA by their student ID.

You have 10,000 students. A sorted array lets you binary-search in O(log n). A linked list is O(n).

Is there a way to look up a value by key in O(1) — without searching at all?

6.1 The Map ADT

A **map** (also called a dictionary or associative array) stores **key–value pairs** and supports lookup by key.

Operation	Description	Goal
`insert(key, value)`	Store a new key–value pair	O(1) avg
`remove(key)`	Delete the pair with the given key	O(1) avg
`search(key)`	Return the value associated with key	O(1) avg
`contains(key)`	True if the key exists	O(1) avg

Keys must be **unique**. Values need not be.

Maps appear everywhere: phone books, symbol tables in compilers, DNS lookups, caches, language dictionaries (`{"hello": "bonjour"}`).

Map ADT vs. Other Structures

Structure	Search	Insert	Delete	Ordered?
Unsorted array	O(n)	O(1) amort.	O(n)	No
Sorted array	O(log n)	O(n)	O(n)	Yes
BST (balanced)	O(log n)	O(log n)	O(log n)	Yes
Hash table	**O(1) avg**	**O(1) avg**	**O(1) avg**	**No**

The hash table achieves average O(1) for all three core operations — a dramatic win over every sorted structure.

> The trade-off: hash tables give up **ordering**. You cannot iterate keys in sorted order without additional work.

Part 2: Hash Tables

Section 6.2

🤔 How can you retrieve a value in O(1)?

Arrays give O(1) access — but only by integer index.

If your keys are arbitrary (strings, IDs, objects) — you can't index directly.

What if you could convert any key into a valid array index? What would that function need to guarantee?

6.2 Hash Tables

A **hash table** stores key–value pairs in an array by computing an index from the key.

```
hash_function(key)  →  index  →  table[index] = value
```

The two-step process:

1. Apply a **hash function** `h(key)` to get a raw integer
2. Map it into the table: `index = h(key) % table_size`

```
key = "alice"
h("alice") = 92318451
index = 92318451 % 10 = 1
table[1] = value
```

The entire lookup is O(1) — no search, no comparison loop — just arithmetic.

The Collision Problem

Two different keys can hash to the same index. This is called a **collision**.

```
h("alice") % 10 = 1
h("carol") % 10 = 1   ← collision!
```

Collisions are **unavoidable** when the key space is larger than the table (and it always is). The hash table must have a strategy for handling them.

The two main collision resolution strategies are:

- **Chaining** — store a list of all items that hash to the same slot
- **Open addressing** — find another empty slot in the same table

Load Factor

The **load factor** λ (lambda) measures how full the table is:

```
λ = number of items / table size
```

Load Factor	Meaning
λ = 0.0	Empty table
λ = 0.5	Half full — few collisions expected
λ = 0.9	Nearly full — many collisions, degraded performance
λ > 1.0	Only possible with chaining

Rule of thumb: keep λ below **0.7** for open addressing, and below **1.0–2.0** for chaining. Past these thresholds, resize the table.

Part 3: Chaining

Section 6.3

🤔 What's the simplest way to handle multiple items landing in the same slot?

Each slot in the array currently holds one item. Two items can't fit.

What if each slot held a list instead of a single item?

6.3 Chaining

In **separate chaining**, each slot holds a linked list of all key–value pairs that hash to that index.

```
table[0]:  → null
table[1]:  → ("alice", 91) → ("carol", 87) → null
table[2]:  → ("bob",   95) → null
table[3]:  → null
...
```

- **Insert**: hash the key, prepend to the list at that slot — O(1)
- **Search**: hash the key, walk the list at that slot — O(λ) average
- **Delete**: hash the key, unlink the node from the list — O(λ) average

When λ is small (< 1), each list is very short — lookups stay near O(1).

Chaining: Performance Analysis

With **uniform hashing** (each key equally likely in any slot):

Operation	Average Case	Worst Case
Insert	$O(1)$	$O(1)$
Search (hit)	$O(1 + \lambda)$	$O(n)$
Search (miss)	$O(1 + \lambda)$	$O(n)$
Delete	$O(1 + \lambda)$	$O(n)$

Worst case occurs when all n keys hash to the **same slot** — the table degenerates to a single linked list.

> With a good hash function and $\lambda < 1$, average-case $O(1)$ is achieved in practice. Worst case is a theoretical concern, not a typical one.

Part 4: Linear Probing

Section 6.4

🤔 Chaining uses extra memory for linked list nodes.

What if you wanted to keep everything inside the array itself — no pointers, no linked lists?

When a collision occurs at index `i`, where could you look next?

What's the simplest possible "next slot" strategy?

6.4 Linear Probing

Open addressing stores all items directly in the table array. On collision, **probe** for the next empty slot.

Linear probing checks consecutive slots:

```
probe sequence: h(key), h(key)+1, h(key)+2, ... (mod table_size)
```

```
Insert "alice" → h("alice") % 7 = 3 → table[3] is empty → place here
Insert "carol" → h("carol") % 7 = 3 → table[3] occupied!
                                      → try table[4] → empty → place here
Insert "dave"  → h("dave")  % 7 = 4 → table[4] occupied!
                                      → try table[5] → empty → place here
```

Linear Probing: The Clustering Problem

Linear probing suffers from **primary clustering** — long runs of occupied slots build up and attract more collisions.

```
[ ][ ][X][X][X][X][X][ ][ ]
         ↑ ↑ ↑ ↑ ↑
         cluster grows with every new collision
```

A key hashing anywhere into the cluster must probe to the end of it.

> Primary clustering degrades performance significantly as $\lambda \to 0.7$. The expected number of probes for a **miss** is approximately $\frac{1}{2}(1 + 1/(1-\lambda)^2)$.

Linear Probing: Deletion Problem

You **cannot** simply mark a deleted slot as empty. A search that passes through it will stop too early and report a false miss.

The fix: mark deleted slots with a **tombstone** sentinel.

```
Probe sequence for "carol":  [3] → [4] → [5]

If table[4] is deleted and marked empty:
  search for "carol" finds empty at [4] and wrongly returns "not found"

If table[4] has a tombstone:
  search skips it and continues to [5] ✓
```

Tombstones can be reused on future insertions. Too many tombstones degrade performance and require periodic rehashing.

Part 5: Double Hashing

Section 6.5

🤔 **Linear probing clusters because every key at the same slot uses the same probe step (+1).**

What if different keys used different step sizes?

How could you derive a per-key step size from the key itself?

6.5 Double Hashing

Double hashing uses a second hash function to compute the probe step, eliminating primary clustering.

```
probe(i) = (h1(key) + i * h2(key)) % table_size
```

Two keys that collide at `h1` will take different paths if `h2` gives them different steps.

```
h1("alice") % 11 = 3,  h2("alice") % 11 = 4
→ probes: 3, 7, 0, 4, 8, 1, ...

h1("carol") % 11 = 3,  h2("carol") % 11 = 7
→ probes: 3, 10, 6, 2, 9, 5, ...
```

The two keys diverge immediately after their initial collision — no clustering.

Choosing h2

`h2` must satisfy two constraints:

1. **Never return 0** — a step of 0 means infinite loop on the same slot
2. **Be coprime with table_size** — ensures the probe visits every slot

A classic choice when table size is prime:

```
int h1(int key) { return key % TABLE_SIZE; }
int h2(int key) { return 1 + (key % (TABLE_SIZE - 1)); }
```

The `1 +` guarantees a nonzero step. Using a prime table size ensures coprimality.

> Double hashing is the best-performing open-addressing strategy. It approaches the theoretical ideal of **uniform probing**.

Open Addressing Strategies Compared

Strategy	Probe Sequence	Clustering	Performance
Linear probing	h, h+1, h+2, …	Primary (bad)	Fast cache
Quadratic probing	h, h+1, h+4, h+9, …	Secondary (moderate)	Moderate
Double hashing	h1, h1+h2, h1+2h2, …	None	Best avg probes

All open-addressing methods require $\lambda < 1$ (table can never be completely full).

Open Addressing Strategies Compared

Linear probing has the best **cache performance** despite its clustering — consecutive slots are cache-friendly. Double hashing has better probe counts but poorer cache behavior.

Part 6: Common Hash Functions

Section 6.6

🤔 What makes a hash function "good"?

Two keys that are very similar — `"cat"` and `"bat"` — should land in completely different slots.

What properties would you want a hash function to have to minimize collisions?

6.6 Common Hash Functions

A good hash function should be:

- **Deterministic** — same key always gives same hash
- **Fast** — ideally O(1) or O(key length)
- **Uniform** — distributes keys evenly across all slots
- **Sensitive** — small changes in key produce large changes in hash

Hashing Strings: Polynomial Rolling Hash

Strings require combining character values without simply summing (anagrams would collide).

```
int hash(const string& s) {
    int h = 0;
    for (char c : s)
        h = h * 31 + c;   // 31 is a small prime
    return abs(h) % TABLE_SIZE;
}
```

Hashing Strings: Polynomial Rolling Hash

```
"cat":  0 * 31 + 'c'(99)  = 99
        99 * 31 + 'a'(97) = 3166
        3166 * 31 + 't'(116) = 98262

"tac":  produces a completely different value ✓
```

The multiplier (31, 37, 53) should be a small prime for good distribution. Java's `String.hashCode()` uses 31.

Part 8: Hashing in Cryptography & Password Storage

Section 6.8

🤔 A website stores user passwords. If the database is stolen, what happens?

Storing passwords in plaintext is obviously dangerous. But even hashing them with a fast hash function has a critical flaw.

What attack becomes easy if an attacker can compute billions of hashes per second?

6.8 Cryptographic Hash Functions

Cryptographic hashes must have **preimage** and **collision** resistance.

Common cryptographic hash functions:

- **SHA-256** — 256-bit output, used in TLS, Git, Bitcoin
- **SHA-3** — NIST standard since 2015
- **MD5** — broken, do not use for security

Password Hashing

Storing `hash(password)` is not sufficient. An attacker can **precompute** hashes for every common password (a **rainbow table** attack).

The fix: add a unique random **salt** to each password before hashing.

```
stored = hash(password + salt)
```

If two users have the same password, their stored hashes differ because their salts differ.

Password Hashing

Purpose-built password hashing algorithms are deliberately expensive:

- **bcrypt** — adjustable work factor (cost parameter)
- **Argon2** — winner of the 2015 Password Hashing Competition; memory-hard
- **scrypt** — memory-hard, resists GPU attacks

> Never use raw SHA-256 or MD5 for passwords. Use bcrypt or Argon2.

Git and Content Hashing

Git uses SHA-1 (and now SHA-256) to identify every object in its database.

```
$ echo "hello" | git hash-object --stdin
ce013625030ba8dba906f756967f9e9ca394464a
```

Every **commit**, **file (blob)**, and **directory (tree)** has a hash. Two files with identical content get the same hash — enabling deduplication.

```
commit abc123  → points to tree def456
tree def456    → lists blob gh789 ("main.cpp"), blob ij012 ("README.md")
```

This is **content-addressed storage** — the hash *is* the identity.

> If even one byte changes, the hash changes completely. Git uses this to detect corruption and guarantee integrity.

Part 9: Hash Table Resizing

Section 6.9

🤔 Your hash table starts with 16 slots. After 12 insertions, $\lambda = 0.75$.

Performance is degrading. You decide to resize to 32 slots.

Can you just copy the array? Why or why not?

6.9 Hash Table Resizing

When λ exceeds a threshold, the table must be **rehashed**:

1. Allocate a new, larger array (typically **2× the current size**, rounded to prime)
2. Re-insert every existing key–value pair into the new array
3. Deallocate the old array

You **cannot** just copy slots — the indices change when the table size changes.

Rehashing Cost and Amortization

Rehashing n items into a new table costs O(n). But it happens infrequently enough that the amortized cost per insertion stays O(1).

Total rehash work after n insertions: 1 + 2 + 4 + … + n = O(n).

Spread over n insertions → **O(1) amortized per insert**.

> This is the same doubling argument as dynamic arrays. The key insight: the expensive operations are rare and geometrically spaced.

When to Resize

Strategy	Trigger
Chaining	$\lambda > 1.0$ (or 0.75 for performance)
Linear probing	$\lambda > 0.5$–0.7
Double hashing	$\lambda > 0.7$

For open addressing, keeping λ below 0.5 is more aggressive but maximizes performance.

> `std::unordered_map` defaults to a **max load factor of 1.0** and rehashes automatically. You can call `reserve(n)` to pre-allocate capacity and avoid rehashing entirely if you know n in advance.

Part 10: Direct Hashing

Section 6.10

🤔 What if every possible key is a small integer in a known range?

Keys are integers from 0 to 999. You have at most 1000 items.

Do you even need a hash function?

6.10 Direct Hashing

Direct hashing (also called a direct-address table) uses the key itself as the index — no hash function at all.

```
table[key] = value
```

```
int grades[1000];          // key = student ID 0–999

grades[412] = 91;          // insert: O(1)
int g = grades[412];       // lookup: O(1) — single array access
grades[412] = -1;          // delete: O(1) (use sentinel or bool array)
```

Every operation is **exactly O(1)** — not amortized, not average-case.

Direct Hashing: Constraints + Trade-offs

Direct hashing is the fastest possible map — but only works under strict conditions.

Requirements:

- Keys must be **non-negative integers** (or trivially mapped to them)
- Key range must be **known and bounded**
- Key range must be **small enough** to fit in memory

Direct Hashing: Constraints + Trade-offs

Scenario	Direct Hash?	Hash Table?
Student IDs 0–9999, ≤5000 students	✅ Ideal	✅ Works
Phone numbers (10 digits)	❌ 10 billion slots	✅ Works
String keys	❌ Not integers	✅ Works
IP addresses (32-bit)	❌ 4 billion slots	✅ Works

> Direct hashing is the concept behind **counting sort** and **bucket sort**. When the key range is small and dense, it outperforms everything else.

Direct Hashing: Real Applications

Counting sort — O(n + k) sorting when k (key range) is small:

```
int count[256] = {};          // direct hash on ASCII values
for (char c : input) count[c]++;
for (int i = 0; i < 256; i++)
    while (count[i]--) output.push_back(i);
```

Visited set in BFS — when nodes are integers 0..n–1:

```
bool visited[NUM_NODES] = {};  // O(1) lookup, no hash function needed
```

Character frequency tables — a direct-address table over the 256 ASCII values.

> Any time your keys are small dense integers, reach for a direct-address table first. You get O(1) guaranteed, with no collisions, no hash function, and no resizing needed.

Lab 5 Wednesday! 😄 Have a nice day!